



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Laboratorio S09

Mensajería con RabbitMQ

MATERIA

Sistemas Distribuidos

Alumno

Reyes Vázquez Fernando

PROFESOR

Oscar Manuel Ruiz Hurtado

FECHA DE ENTREGA | 24 DE OCTUBRE DE 2026

GRUPO 02 | SEMESTRE 2027-1

1. El problema

Para comenzar levanté los servicios de Mexi Banco y creé dos cuentas de prueba: Ana con $1,000 y Beto con $500. Después detuve notificacion y envié una transferencia de $50 para observar qué ocurría cuando no estaba disponible el servicio que genera los avisos.

```
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % curl -i -X POST http://localhost:8080/api/cuentas \
  -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000001","titular":"Ana","saldoInicial":1000}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: b96fd530e40c
Content-Type: application/json
Date: Sat, 24 Oct 2026 01:29:30 GMT

{"id":1,"clabe":"002180000000000001","titular":"Ana","saldo":1000}%
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % curl -i -X POST http://localhost:8080/api/cuentas \
  -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000002","titular":"Beto","saldoInicial":500}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: b96fd530e40c
Content-Type: application/json
Date: Sat, 24 Oct 2026 01:29:44 GMT

{"id":2,"clabe":"002180000000000002","titular":"Beto","saldo":500}%
```

Figura 1. *Cuentas de Ana y Beto creadas con saldos iniciales de $1,000 y $500.*

La petición devolvió un error 500. Sin embargo, al consultar la transferencia y los saldos encontré que el movimiento de dinero sí había ocurrido: Ana quedó con $950, Beto con $550 y la transferencia 1 se registró en estado ABONADA, no COMPLETADA.

```
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % docker compose stop notificacion
[+] stop 1/1
 ✔ Container mexi-banco-s09-notificacion-1 Stopped
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50}'
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Sat, 24 Oct 2026 01:35:04 GMT

{"timestamp":"2026-10-24T01:35:04.387+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}%
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % curl -s 'http://localhost:8080/api/transferencias?estado=ABONADA'
[{"id":1,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50.00,"fecha":"2026-10-24T01:35:01.272152Z","estado":"ABONADA","motivo":null,"saldoOrigenTrasCargo":950.00,"intentosDeCompensacion":0,"actualizada":"2026-10-24T01:35:03.808488Z"}]
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % curl -s http://localhost:8080/api/cuentas/002180000000000001
{"id":1,"clabe":"002180000000000001","titular":"Ana","saldo":950.00}%
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % curl -s http://localhost:8080/api/cuentas/002180000000000002
{"id":2,"clabe":"002180000000000002","titular":"Beto","saldo":550.00}%
```

Figura 2. *Error al apagar notificacion y consulta posterior del estado ABONADA y de ambos saldos.*

¿Por qué el cliente recibe un error?

La transferencia todavía hacía una llamada HTTP síncrona para notificar. Aunque el cargo y el abono ya se habían realizado, la falta de respuesta de notificacion impedía terminar la solicitud. El error que recibió el cliente no significaba que el dinero no se hubiera movido; el problema era que una tarea secundaria seguía afectando a la operación principal.

2. Publicar eventos

Identificar cada evento

Agregué el atributo eventoId a Transferencia, generado con UUID. Lo conservé en el registro para identificar un evento aunque vuelva a publicarse. Después creé EventoTransferencia.java como la representación del mensaje, con tipo, identificador, cuentas, monto, saldos y fecha. Se contemplan TransferenciaRealizada y TransferenciaRevertida.

```java
// Transferencia.java M
// transferencia > src > main > java > mx > mexibanco > transferencia > Transferencia.java
public class Transferencia {
    @Column
    private Instant actualizada;

    @Column(unique = true)
    private UUID eventoId;

    protected Transferencia() {
    }

    public Transferencia(String claveOrigen, String claveDestino, BigDecimal monto) {
        this.claveOrigen = claveOrigen;
        this.claveDestino = claveDestino;
        this.monto = monto;
        this.fecha = Instant.now();
        this.estado = INICIADA;
        this.intentosDeCompensacion = 0;
        this.actualizada = this.fecha;
        this.eventoId = UUID.randomUUID();
    }

    @PostLoad
    void asegurarEventoId() {
        if (eventoId == null) {
            eventoId = UUID.randomUUID();
        }
    }
```

Figura 3. *Nuevo campo eventoId y generación de su identificador en Transferencia.java.*

```java
package mx.mexibanco.mensajeria;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.UUID;

public record EventoTransferencia(UUID eventoId, String tipo, Long transferenciaId, String clabeOrigen,
                                  String clabeDestino, BigDecimal monto, BigDecimal saldoOrigen,
                                  BigDecimal saldoDestino, Instant fecha) {

    public static final String REALIZADA = "TransferenciaRealizada";
    public static final String REVERTIDA = "TransferenciaRevertida";

    /** La saga llegó a COMPLETADA: el dinero salió de origen y llegó a destino. */
    public static EventoTransferencia realizada(UUID eventoId, Long transferenciaId, String clabeOrigen, String clabeDestino,
                                                BigDecimal monto, BigDecimal saldoOrigen, BigDecimal saldoDestino, Instant fecha) {
        return new EventoTransferencia(eventoId, REALIZADA, transferenciaId, clabeOrigen, clabeDestino, monto,
            saldoOrigen, saldoDestino, fecha);
    }

    /** La saga llegó a COMPENSADA: el abono falló y el cargo ya regresó. Destino no se tocó: sin saldoDestino. */
    public static EventoTransferencia revertida(UUID eventoId, Long transferenciaId, String clabeOrigen, String clabeDestino,
                                                BigDecimal monto, BigDecimal saldoOrigenTrasDevolucion, Instant fecha) {
        return new EventoTransferencia(eventoId, REVERTIDA, transferenciaId, clabeOrigen, clabeDestino, monto,
            saldoOrigenTrasDevolucion, null, fecha);
    }

    /** La routing key del contrato. */
    public String routingKey() {
        return REVERTIDA.equals(tipo) ? "transferencia.revertida" : "transferencia.realizada";
    }
}
```

Figura 4. *Nueva clase EventoTransferencia.java con los datos que se enviarán a RabbitMQ.*

Publicador y confirmación del broker

Creé ConfiguracionMensajeria.java para declarar el exchange topic transferencias y configurar el envío en JSON. También agregué PublicadorDeEventos.java y EventoNoPublicadoException.java. El publicador envía el evento con RabbitTemplate, asigna su messageId, solicita un publisher confirm y revisa si RabbitMQ regresó el mensaje por no encontrar una cola receptora.

```java
// EventoNoPublicadoException.java U  |  PublicadorDeEventos.java U
// transferencia > src > main > java > mx > mexibanco > mensajeria > EventoNoPublicadoException.java
package mx.mexibanco.mensajeria;

public class EventoNoPublicadoException extends RuntimeException {
    public EventoNoPublicadoException(String mensaje) { super(mensaje); }
    public EventoNoPublicadoException(String mensaje, Throwable causa) { super(mensaje, causa); }
}
```

Figura 5. *Nueva excepción utilizada cuando la publicación de un evento no puede confirmarse.*



```
    private final RabbitTemplate rabbit;
    private final long esperaConfirmacionMs;

    public PublicadorDeEventos(RabbitTemplate rabbit,
                               @Value("${mexibanco.mensajeria.espera-confirmacion-ms:5000}") long esperaConfirmacionMs) {
        this.rabbit = rabbit;
        this.esperaConfirmacionMs = esperaConfirmacionMs;
    }

    /** Regresa solo si el broker confirmó el evento. Si no, lanza EventoNoPublicadoException. */
    public void publicar(EventoTransferencia evento) {
        String id = evento.eventoId().toString();
        CorrelationData correlacion = new CorrelationData(id);
        try {
            rabbit.convertAndSend(ConfiguracionMensajeria.EXCHANGE, evento.routingKey(), evento, mensaje -> {
                mensaje.getMessageProperties().setDeliveryMode(MessageDeliveryMode.PERSISTENT); // a disco
                mensaje.getMessageProperties().setMessageId(id);   // se ve en el panel sin abrir el JSON
                return mensaje;
            }, correlacion);
        } catch (RuntimeException sinConexion) {
            throw new EventoNoPublicadoException("No se pudo mandar el evento " + id + " al broker: " + sinConexion.getMessage(), sinConexion);
        }
        CorrelationData.Confirm confirmacion;
        try {
            confirmacion = correlacion.getFuture().get(esperaConfirmacionMs, TimeUnit.MILLISECONDS);
        } catch (TimeoutException | ExecutionException sinRespuesta) {
            throw new EventoNoPublicadoException("El broker no confirmo el evento " + id + ": " + sinRespuesta, sinRespuesta);
        } catch (InterruptedException interrumpido) {
            Thread.currentThread().interrupt();
            throw new EventoNoPublicadoException("Se interrumpio la espera del evento " + id, interrumpido);
        }
        if (!confirmacion.isAck()) {
            throw new EventoNoPublicadoException("El broker rechazo el evento " + id + " (nack): " + confirmacion.getReason());
        }
        ReturnedMessage regresado = correlacion.getReturned();   // lo regresó por mandatory
        if (regresado != null) {
            throw new EventoNoPublicadoException("Ninguna cola recibio el evento " + id + " con la routing key "
                + regresado.getRoutingKey() + " (" + regresado.getReplyText() + ")");
        }
    }
```

Figura 6. *Código principal de PublicadorDeEventos.java: envío, espera del confirm y revisión de mensajes retornados.*

Fragmento clave: comprobación del confirm

```
confirmacion = correlacion.getFuture()
    .get(esperaConfirmacionMs, TimeUnit.MILLISECONDS);
if (!confirmacion.isAck()) { /* publicación no confirmada */ }
if (correlacion.getReturned() != null) { /* sin ruta */ }
```

Publicar después de guardar

En TransferenciaService sustituí las llamadas HTTP a notificacion por la publicación de eventos. En el caso exitoso, primero se marca y guarda COMPLETADA y después se publica TransferenciaRealizada. Cuando se compensa el cargo, se publica TransferenciaRevertida después de guardar COMPENSADA. La publicación se hace en avisar(), donde una falla se registra en los logs sin deshacer la transferencia.



```
ventoNoPublicadoException.java U   TransferenciaService.java M ×   PublicadorDeEventos.java U
sferencia > src > main > java > mx > mexibanco > transferencia > TransferenciaService.java
    public class TransferenciaService {
        public Transferencia transferir(String claveOrigen, String claveDestino, BigDecimal monto) {
            movimientos.registrar(t.clave("asiento-cargo"), claveOrigen,
                TipoMovimiento.TRANSFERENCIA_ENVIADA, monto.negate(), t.getSaldoOrigenTrasCargo(), "a " + claveDestino);
            movimientos.registrar(t.clave("asiento-abono"), claveDestino,
                TipoMovimiento.TRANSFERENCIA_RECIBIDA, monto, destino.saldo(), "de " + claveOrigen);
            // aquí estaban notificaciones.notificar(cargo) y notificaciones.notificar(abono): bórralas

            t.marcarCompletada();
            t = guardar(t);                                  // primero la base dice COMPLETADA
            avisar(EventoTransferencia.realizada(t.getEventoId(), t.getId(), claveOrigen, claveDestino, monto,
                t.getSaldoOrigenTrasCargo(), destino.saldo(), t.getActualizada()));   // luego el recado
            return t;
        }
```

Figura 7. *TransferenciaService.java publica TransferenciaRealizada después de guardar COMPLETADA.*

¿Qué garantiza un publisher confirm?

El publisher confirm permite saber si RabbitMQ aceptó la publicación del mensaje. No prueba que notificacion lo haya consumido, que se haya guardado un aviso ni que la operación de base de datos y la publicación sean atómicas. Durante una prueba, una transferencia quedó COMPLETADA, pero el log mostró NO_ROUTE porque todavía no había una cola enlazada al exchange.



Figura 8. *Transferencia completada y registro NO_ROUTE cuando el evento aún no tenía una cola receptora.*

Incidencia durante las pruebas

También encontré un timeout al intentar transferir $50: la API respondió 503 y la transferencia se registró como FALLIDA, aunque PostgreSQL mostró que el cargo sí se había aplicado. Para continuar dejé una instancia de cuenta, detuve spei y aumenté el tiempo de lectura a 10 segundos. Con esos ajustes conseguí nuevas transferencias 201 Created, pero el cargo anterior quedó pendiente de reconciliación. Un timeout de comunicación no demuestra que la operación remota no se ejecutó.

Los ajustes se aplicaron desde la Terminal con los siguientes comandos:

```
docker compose up -d --scale cuenta=1 cuenta
docker compose stop spei
TIMEOUT_LECTURA_MS=10000 docker compose up -d --no-deps --force-recreate transferencia
docker compose exec transferencia printenv TIMEOUT_LECTURA_MS
```

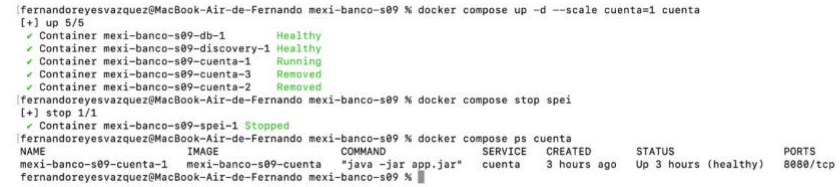


Figura 8a. *Reducción del servicio cuenta a una instancia y detención de spei durante la corrección del timeout.*

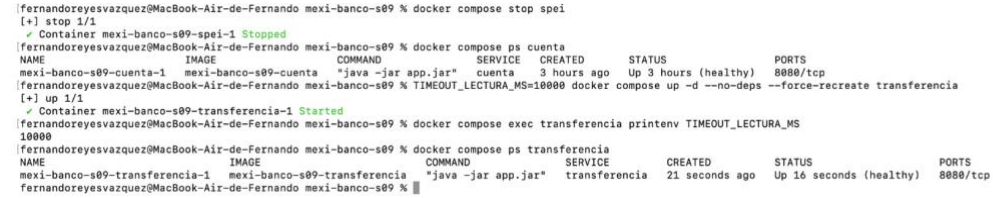


Figura 8b. *Reinicio de transferencia con lectura de 10 000 ms y comprobación de la variable aplicada.*

3. Desacople en tiempo

Transferencias mientras notificacion estaba apagado

En notificacion creé EventoTransferencia.java, ConfiguracionMensajeria.java y ConsumidorDeTransferencias.java. La cola durable notificacion.transferencias se enlazó al exchange transferencias usando transferencia.*. El consumidor transforma cada mensaje a objeto Java y, una vez guardados los avisos, envía basicAck(). Se configuró el modo de confirmación manual.

```java
package mx.mexibanco.notificacion.mensajeria;

import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Declarables;
import org.springframework.amqp.core.ExchangeBuilder;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.amqp.core.TopicExchange;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class ConfiguracionMensajeria {

    public static final String EXCHANGE = "transferencias";
    public static final String COLA = "notificacion.transferencias";
    public static final String PATRON = "transferencia.*";

    @Bean
    Declarables topologia() {
        // El mismo exchange que declara transferencia, con las mismas propiedades (topic, durable).
        // Se declara aquí también para poder enlazar la cola aunque notificacion arranque primero.
        TopicExchange transferencias = ExchangeBuilder.topicExchange(EXCHANGE).durable(true).build();
        // Durable: la cola y sus mensajes persistentes sobreviven a un reinicio del broker.
        Queue cola = QueueBuilder.durable(COLA).build();
        return new Declarables(transferencias, cola, BindingBuilder.bind(cola).to(transferencias).with(PATRON));
    }
}
```

Figura 9. *Nueva configuración de la cola principal y su binding con el exchange transferencias.*

ConfiguracionMensajeria.java U | EventoTransferencia.java U | NotificacionService.java

notificacion > src > main > java > mx > mexibanco > notificacion > EventoTransferencia.java

```java
package mx.mexibanco.notificacion;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.UUID;

public record EventoTransferencia(UUID eventoId, String tipo, Long transferenciaId, String clabeOrigen,
                                  String clabeDestino, BigDecimal monto, BigDecimal saldoOrigen,
                                  BigDecimal saldoDestino, Instant fecha) {

    public static final String REALIZADA = "TransferenciaRealizada";
    public static final String REVERTIDA = "TransferenciaRevertida";
}
```

Figura 10. *Nueva clase EventoTransferencia.java del servicio notificacion.*

Detuve notificacion y envié tres transferencias de $11, $12 y $13. Las tres solicitudes respondieron 201 y RabbitMQ mostró tres mensajes Ready, aunque el servicio encargado de generar los avisos seguía apagado.

```
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % docker compose stop notificacion
[+] stop 1/1
 ✔ Container mexi-banco-s09-notificacion-1 Stopped
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % for m in 11 12 13; do
  curl -s -o /dev/null -w '%{http_code}\n' -X POST http://localhost:8080/api/transferencias \
    -H 'Content-Type: application/json' \
    -d "{\"claveOrigen\":\"002180000000000001\",\"claveDestino\":\"002180000000000002\",\"monto\":$m}"
done
201
201
201
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias     3       0
```

Figura 11. *Tres transferencias exitosas con el consumidor detenido; RabbitMQ conservó 3 mensajes Ready.*

Queues

All queues (1)

Pagination

Page 1 of 1 - Filter: Regex ?

Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	notificacion.transferencias	classic	D Args	running	3	0	3	0.00/s			

Add a new queue

Figura 12. *Vista de RabbitMQ con los tres mensajes en espera en la cola de notificaciones.*

Al volver a iniciar notificacion, la cola quedó en cero y los tres eventos generaron sus avisos. La bandeja mostró un aviso de salida para Ana y otro de entrada para Beto por cada transferencia. Los registros del consumidor mostraron dos avisos nuevos para cada evento.

```
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % docker compose start notificacion
[+] start 4/4
 ✔ Container mexi-banco-s09-db-1           Healthy                                                
```

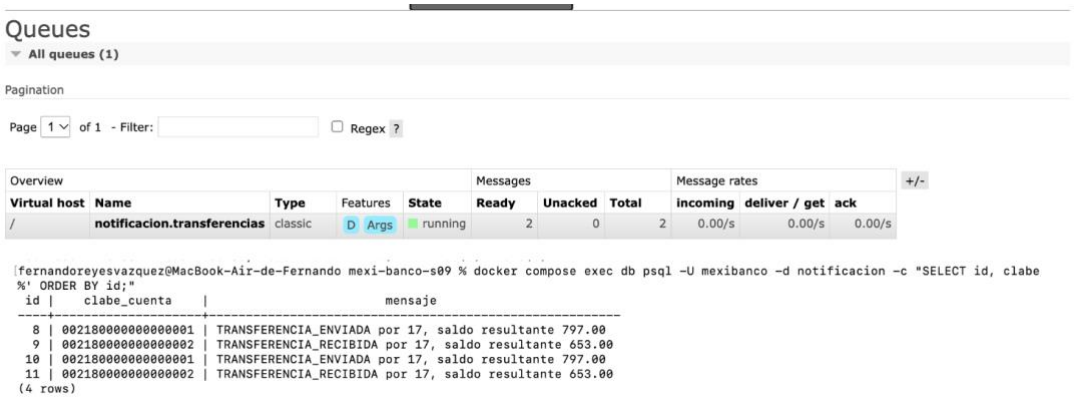


Figura 15. *Reenvío manual del evento de $17 y observación de las copias en la cola.*

```
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % docker compose exec db psql -U mexibanco -d notificacion -c "SELECT id
%' ORDER BY id;"
 id |    clabe_cuenta     |                          mensaje
----+---------------------+-------------------------------------------------------------
  8 | 002180000000000001 | TRANSFERENCIA_ENVIADA por 17, saldo resultante 797.00
  9 | 002180000000000002 | TRANSFERENCIA_RECIBIDA por 17, saldo resultante 653.00
 10 | 002180000000000001 | TRANSFERENCIA_ENVIADA por 17, saldo resultante 797.00
 11 | 002180000000000002 | TRANSFERENCIA_RECIBIDA por 17, saldo resultante 653.00
(4 rows)
```

Figura 15a. *Consulta de la tabla notificacion: cuatro filas con id, CLABE y mensaje repetido por $17.*

Arreglo con eventoId e inserción única

Para corregirlo creé EventoProcesado.java y EventoProcesadoRepository.java en el servicio notificacion. La tabla evento_procesado usa evento_id como llave primaria. Al recibir un evento, se intenta registrarlo con INSERT ... ON CONFLICT DO NOTHING; si ese identificador ya existe, el método no guarda más avisos. El registro del evento y las notificaciones se realizan en la misma transacción local.

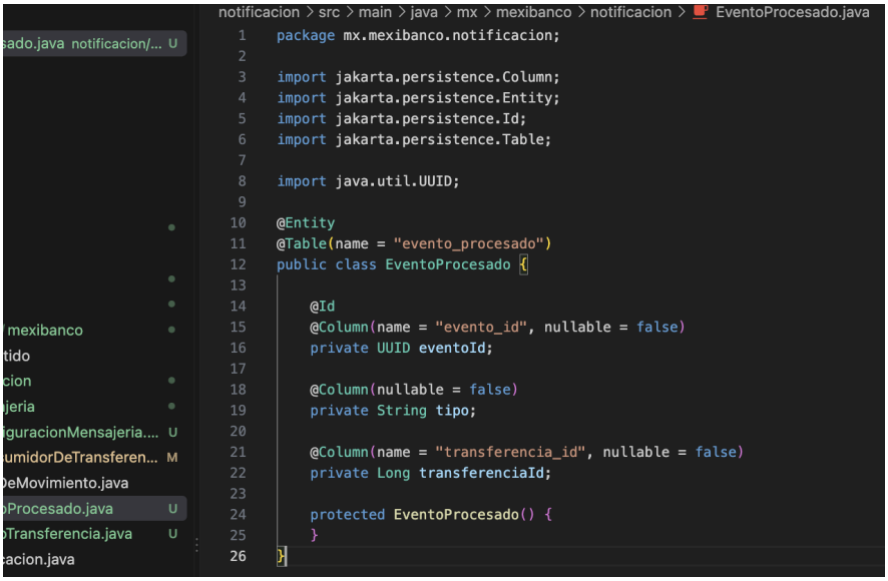
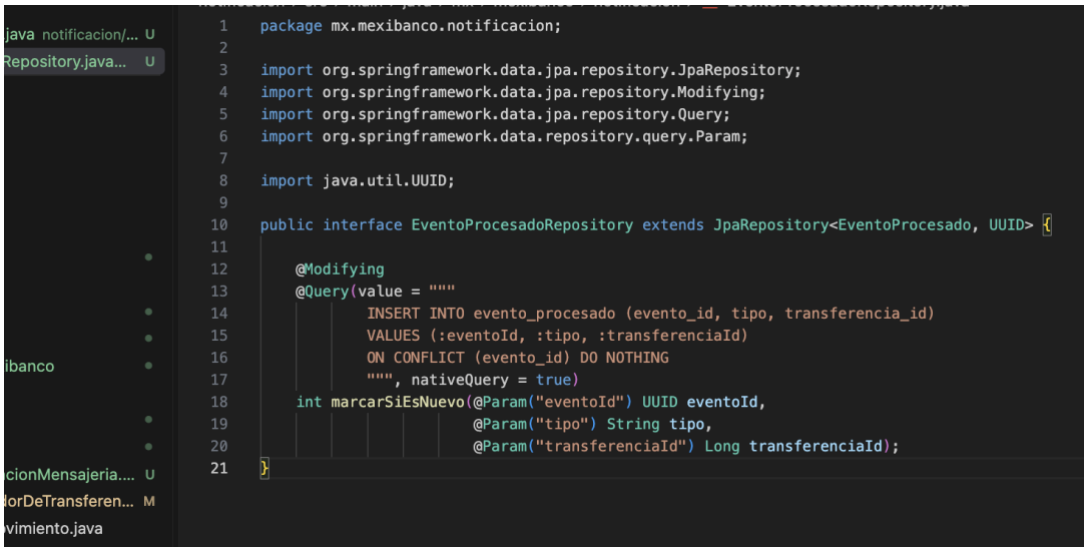


Figura 16. *Nueva entidad EventoProcesado.java con evento_id como identificador único.*



```java
package mx.mexibanco.notificacion;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.UUID;

public interface EventoProcesadoRepository extends JpaRepository<EventoProcesado, UUID> {

    @Modifying
    @Query(value = """
            INSERT INTO evento_procesado (evento_id, tipo, transferencia_id)
            VALUES (:eventoId, :tipo, :transferenciaId)
            ON CONFLICT (evento_id) DO NOTHING
            """, nativeQuery = true)
    int marcarSiEsNuevo(@Param("eventoId") UUID eventoId,
                        @Param("tipo") String tipo,
                        @Param("transferenciaId") Long transferenciaId);
}
```

Figura 17. *Nuevo repositorio que registra el evento una sola vez mediante ON CONFLICT DO NOTHING.*

Fragmento clave: evitar el segundo efecto

```
int insertados = eventosProcesados.marcarSiEsNuevo(
    evento.eventoId(), evento.tipo(), evento.transferenciaId());
if (insertados == 0) {
    return List.of();
}
```

Segunda prueba: sin avisos duplicados

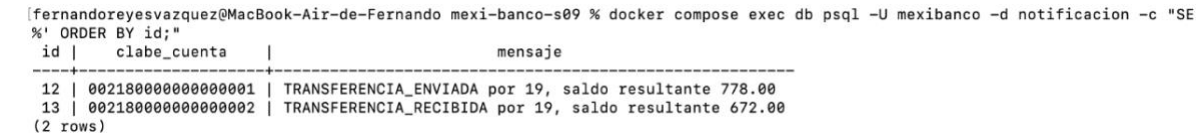
Repetí el experimento con la transferencia 9 por $19, reenviando exactamente el mismo eventoId. El primer consumo generó dos avisos y el segundo registró que el evento ya estaba procesado, por lo que generó cero. PostgreSQL mostró una sola fila en evento_procesado y únicamente dos notificaciones, con los saldos $778 para Ana y $672 para Beto.



```
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % docker compose start notificacion
[+] start 4/4
 ✔ Container mexi-banco-s09-db-1          Healthy                                                                                                0.5s
 ✔ Container mexi-banco-s09-discovery-1   Healthy                                                                                                0.5s
 ✔ Container mexi-banco-s09-rabbitmq-1    Healthy                                                                                                0.5s
 ✔ Container mexi-banco-s09-notificacion-1 Started                                                                                               0.2s
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias     0       0
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % docker compose logs --tail=100 notificacion | grep -E '\[consumidor\]|ya procesado|ERROR'
notificacion-1  | 2026-10-24T07:46:41.968Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia 9 (evento b4de7336-ca20-49ed-95e6-d7cb1dbab916, redelivered=true): 2 aviso(s) nuevo(s)
notificacion-1  | 2026-10-24T07:46:41.971Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.notificacion.NotificacionService     : [notificacion] Evento b4de7336-ca20-49ed-95e6-d7cb1dbab916 ya procesado, se omiten avisos
notificacion-1  | 2026-10-24T07:46:41.972Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRealizada de la transferencia 9 (evento b4de7336-ca20-49ed-95e6-d7cb1dbab916, redelivered=false): 0 aviso(s) nuevo(s)
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % docker compose exec db psql -U mexibanco -d notificacion -c "SELECT evento_id, tipo, transferencia_id FROM evento_procesado WHERE evento_id = 'b4de7336-ca20-49ed-95e6-d7cb1dbab916';"
              evento_id              |         tipo          | transferencia_id
-------------------------------------+-----------------------+------------------
 b4de7336-ca20-49ed-95e6-d7cb1dbab916 | TransferenciaRealizada |                9
(1 row)

fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % docker compose exec db psql -U mexibanco -d notificacion -c "SELECT id, clabe_cuenta, mensaje FROM notificacion WHERE mensaje LIKE '%por 19,%' ORDER BY id;"
 id |    clabe_cuenta    |                          mensaje
----+--------------------+-------------------------------------------------------------
 12 | 002180000000000001 | TRANSFERENCIA_ENVIADA por 19, saldo resultante 778.00
 13 | 002180000000000002 | TRANSFERENCIA_RECIBIDA por 19, saldo resultante 672.00
(2 rows)
```

Figura 18. *Logs del consumidor y verificación de un evento registrado y dos avisos por $19.*



```
fernandoreyesvazquez@MacBook-Air-de-Fernando mexi-banco-s09 % docker compose exec db psql -U mexibanco -d notificacion -c "SE
%' ORDER BY id;"
 id |    clabe_cuenta    |                          mensaje
----+--------------------+-------------------------------------------------------------
 12 | 002180000000000001 | TRANSFERENCIA_ENVIADA por 19, saldo resultante 778.00
 13 | 002180000000000002 | TRANSFERENCIA_RECIBIDA por 19, saldo resultante 672.00
(2 rows)
```

Figura 18a. *Consulta de la tabla notificacion después del arreglo: solo dos filas por $19.*

Prueba	Primera entrega	Segunda entrega	Avisos totales
Antes: transferencia 8	2	2	4
Después: transferencia 9	2	0	2

¿Por qué el ack se manda después?

Si se confirma antes de guardar, una caída entre el ack y el commit puede hacer que se pierda el aviso sin posibilidad de reentrega. Si se confirma después, RabbitMQ puede repetir el mensaje cuando se pierde la confirmación; por eso es necesaria la idempotencia mediante eventoId. En esta práctica se prefirió poder recibir duplicados y eliminarlos por identificador antes que perder silenciosamente los eventos.

5. La Dead Letter Queue (DLQ)

Mensaje envenenado sin una DLQ

Publiqué el JSON incompleto {"hola": con la routing key transferencia.realizada. Jackson no pudo convertirlo a EventoTransferencia y apareció JsonEOFException. RabbitMQ mostró un mensaje Unacked; al detener notificacion, el mensaje regresó a Ready. Sin un tratamiento específico, un mensaje así puede impedir que la cola avance normalmente.



Figura 19. *Error de deserialización JsonEOFException producido por el JSON incompleto.*

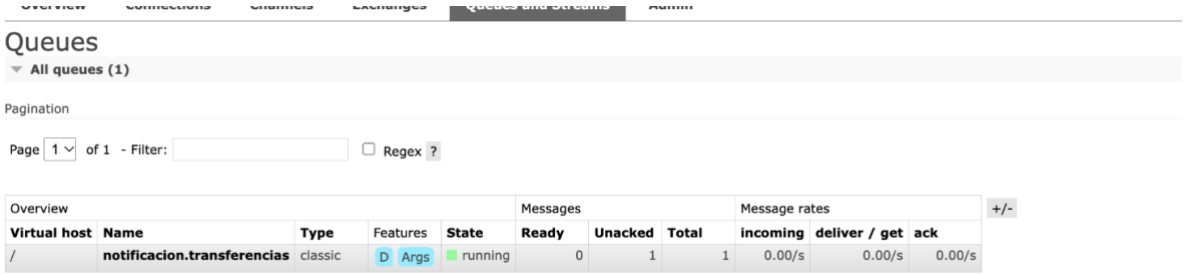


Figura 20. *La cola principal mostró un mensaje Unacked durante el error del consumidor.*

Separar mensajes inválidos

Después agregué el exchange direct transferencias.dlx y la cola durable notificacion.transferencias.dlq. La cola principal quedó configurada con x-dead-letter-exchange y x-dead-letter-routing-key. En ConsumidorDeTransferencias el error de interpretación se captura y se responde con basicNack(etiqueta, false, false), para que RabbitMQ lo rechace sin reencolarlo.

```java
import org.springframework.amqp.core.Declarables;
import org.springframework.amqp.core.DirectExchange;
import org.springframework.amqp.core.ExchangeBuilder;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.amqp.core.TopicExchange;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class ConfiguracionMensajeria {

    public static final String EXCHANGE = "transferencias";
    public static final String COLA = "notificacion.transferencias";
    public static final String PATRON = "transferencia.*";

    @Bean
    Declarables topologia() {
        // Exchange principal para los eventos de transferencia.
        TopicExchange transferencias = ExchangeBuilder.topicExchange(EXCHANGE)
                .durable(true)
                .build();

        // Exchange para los mensajes rechazados.
        DirectExchange dlx = ExchangeBuilder.directExchange("transferencias.dlx")
                .durable(true)
                .build();

        // Cola donde se almacenan los mensajes rechazados.
        Queue dlq = QueueBuilder.durable("notificacion.transferencias.dlq")
                .build();

        // Cola principal con la configuracion de Dead Letter Exchange.
        Queue cola = QueueBuilder.durable(COLA)
                .deadLetterExchange("transferencias.dlx")
                .deadLetterRoutingKey("notificacion.transferencias")
                .build();

        return new Declarables(
                transferencias,
                cola,
                BindingBuilder.bind(cola).to(transferencias).with(PATRON),
                dlx,
                dlq,
                BindingBuilder.bind(dlq).to(dlx).with("notificacion.transferencias")
        );
    }
```

Figura 21. *Configuración de la nueva topología DLQ en ConfiguracionMensajeria.java.*

```java
mponent
ic class ConsumidorDeTransferencias {

private static final Logger log = LoggerFactory.getLogger(ConsumidorDeTransferencias.class);

private final NotificacionService notificaciones;
private final ObjectMapper json;

public ConsumidorDeTransferencias(NotificacionService notificaciones, ObjectMapper json) {
    this.notificaciones = notificaciones;
    this.json = json;
}

@RabbitListener(queues = ConfiguracionMensajeria.COLA)
public void recibir(Message mensaje, Channel canal) throws IOException {
    long etiqueta = mensaje.getMessageProperties().getDeliveryTag();
    boolean reentregado = Boolean.TRUE.equals(mensaje.getMessageProperties().getRedelivered());

    try {
        EventoTransferencia evento = json.readValue(mensaje.getBody(), EventoTransferencia.class);
        List<Notificacion> avisos = notificaciones.avisarDeTransferencia(evento);

        log.info("[consumidor] {} de la transferencia {} (evento {}, redelivered={}): {} aviso(s) nuevo(s)",
                evento.tipo(), evento.transferenciaId(), evento.eventoId(), reentregado, avisos.size());

        canal.basicAck(etiqueta, false);

    } catch (JsonProcessingException error) {
        log.error("[consumidor] Mensaje invalido, se envia a DLQ: {}", error.getOriginalMessage());
        canal.basicNack(etiqueta, false, false);
    }
}
```

Figura 22. *ConsumidorDeTransferencias.java captura JSON inválido y utiliza basicNack sin reencolar.*

La cola anterior se vació y eliminó antes de levantar la nueva configuración, porque RabbitMQ no permite cambiar los argumentos de una cola existente. Después de reconstruir notificacion comprobé que estaban creadas las dos colas y que el binding del exchange de errores apuntaba a la DLQ.



Figura 23. *Argumentos x-dead-letter-exchange y x-dead-letter-routing-key en la cola principal.*

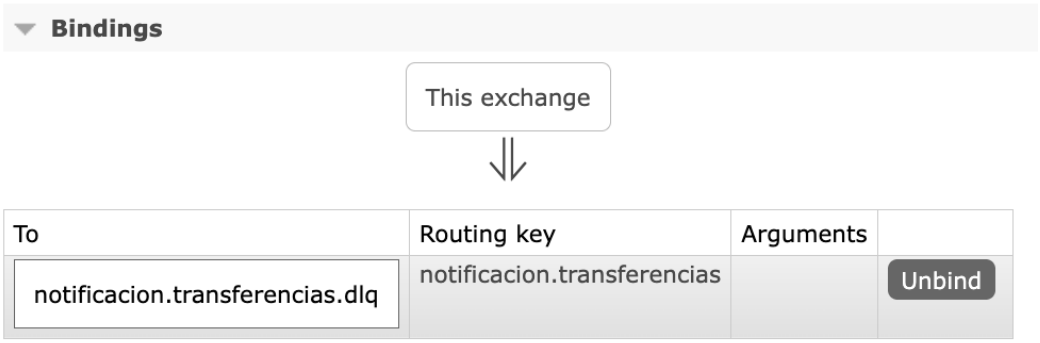


Figura 24. *Binding desde transferencias.dlx hacia notificacion.transferencias.dlq.*

Comprobación del rechazo

Volví a enviar el mismo JSON inválido. Esta vez la cola principal quedó vacía y la DLQ mostró un mensaje Ready. Al inspeccionarlo apareció el encabezado x-death con reason: rejected y count: 1. Los logs también mostraron el rechazo explícito. Esto permitió conservar el mensaje para revisión sin que permaneciera en la cola principal.



Figura 25. *RabbitMQ separó el mensaje inválido: 0 en la cola principal y 1 en la DLQ.*

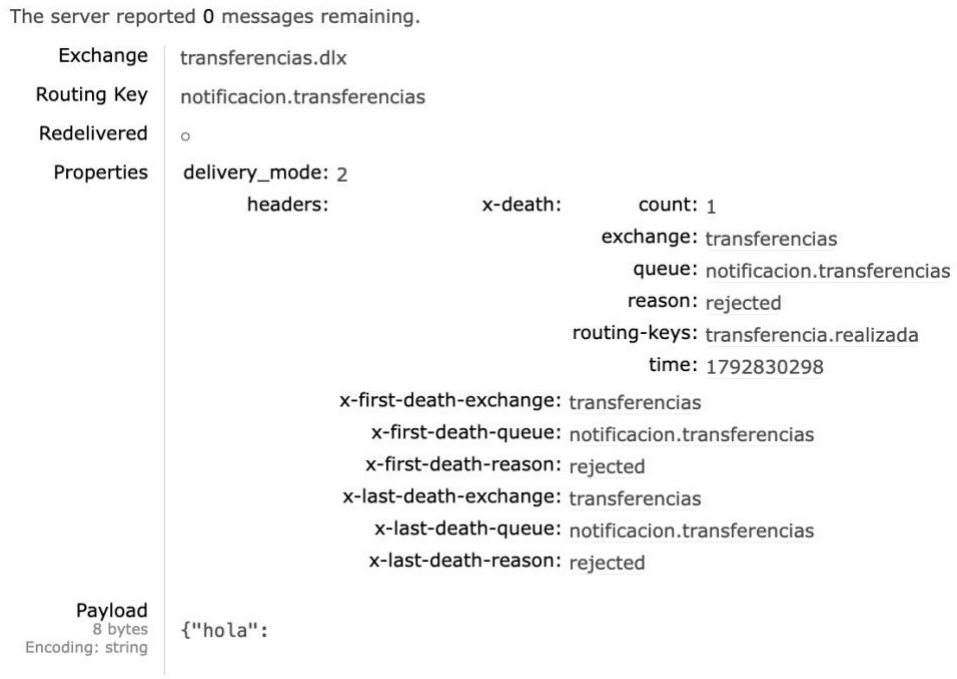


Figura 26. *Encabezado x-death con motivo rejected y payload incompleto del mensaje enviado a la DLQ.*

Un mensaje válido sigue pasando

Por último envié una transferencia de $23, que respondió 201 Created y quedó COMPLETADA con identificador 10. Ana recibió el aviso de salida con saldo $755 y Beto el de entrada con saldo $695. La cola

principal siguió vacía y la DLQ conservó un mensaje, por lo que el error de formato no detuvo las notificaciones normales.



Figura 27. *Transferencia normal por $23 y avisos de ambas cuentas, con el mensaje inválido aún en DLQ.*

6. Preguntas de cierre

Si RabbitMQ cae cuando la saga llega a COMPLETADA

En mi versión primero se guarda el estado COMPLETADA y después se intenta publicar el evento. Si RabbitMQ está caído, la transferencia puede terminar correctamente y el aviso no llegar a notificacion; el error solamente se registra en los logs. El publisher confirm no corrige eso porque confirma la publicación cuando sí hubo comunicación con el broker, pero no guarda automáticamente los eventos pendientes junto con la transferencia. Para cubrir esa separación haría falta otro mecanismo de recuperación.

Por qué el ack va después de guardar

Si enviara el ack antes de guardar, RabbitMQ daría por terminado el mensaje y una caída posterior podría dejar al usuario sin aviso. Al enviarlo después del guardado, si el consumidor falla antes de confirmar, RabbitMQ puede entregarlo otra vez. Esa segunda opción evita reconocer trabajo que todavía no se ha hecho, pero acepta la posibilidad de reentregas; por eso agregué eventoId y la tabla evento_procesado para evitar efectos duplicados.

Elección y sacrificio

Elijo: publicar eventos asíncronos en RabbitMQ en lugar de depender de una llamada HTTP a notificacion. Porque: así la transferencia no necesita esperar a que notificacion esté disponible para mover el dinero. Sacrifico: la notificación inmediata y la certeza de que el aviso ya quedó guardado cuando el cliente recibe la respuesta.